

Processor Scheduling

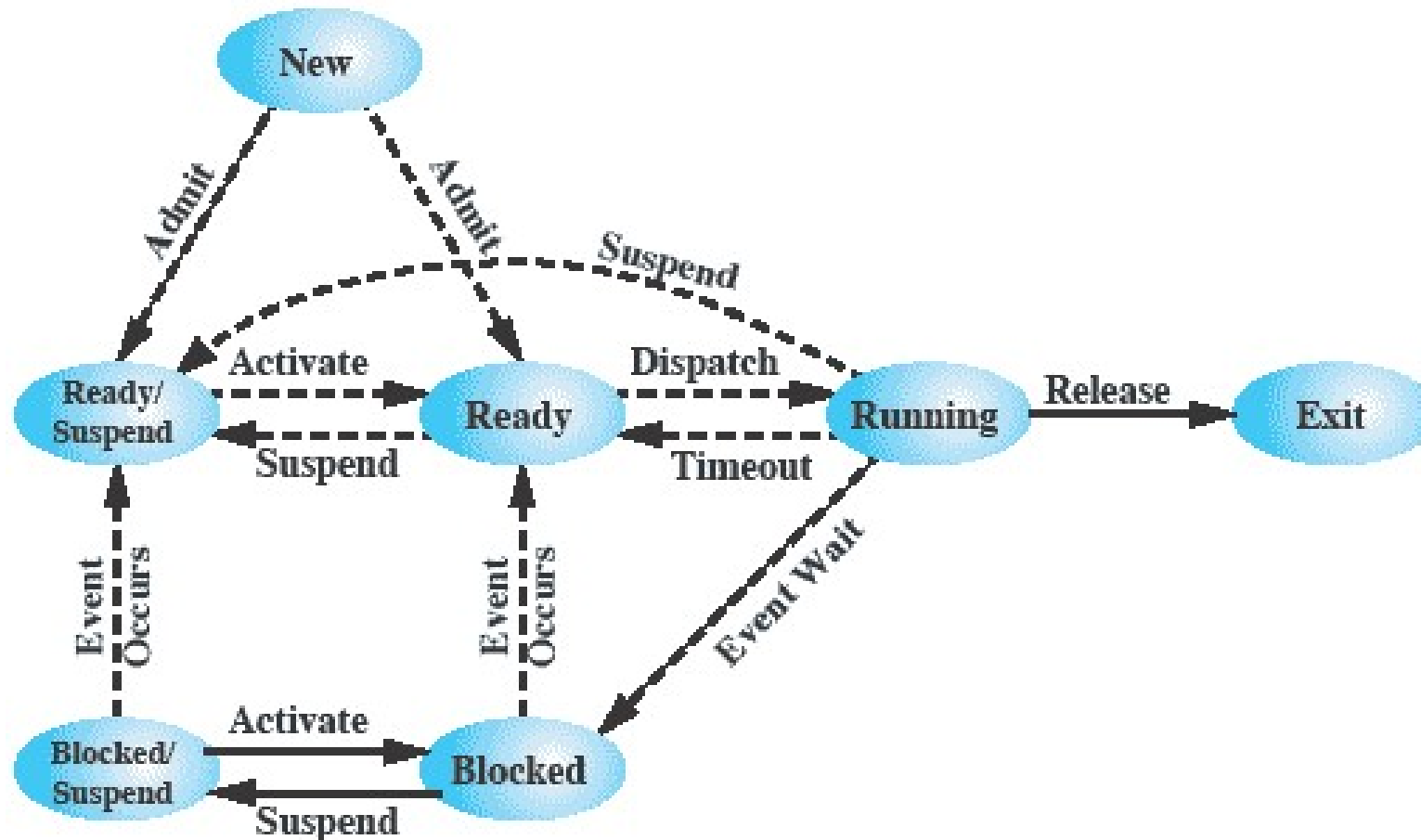


Types of Processor Scheduling

- **Long-term scheduling**: whether to add a new process to the set of processes that are currently active.
 - Controls the degree of multiprogramming
 - ▶ More processes → smaller percentage of time each process is executed
 - ▶ Less processes → CPU will be idle
 - Which process to add?
 - ▶ FCFS, priority, mix I/O-bound/CPU-bound process, balance I/O usage
- **Medium-term scheduling**: whether to add a process partially in main memory and therefore available for execution.
 - Part of the memory management function – swapping
 - Swapping-in decisions are based on the need to manage the performance related to degree of multiprogramming
- **Short-term scheduling**: which ready process to execute next
 - Execute more frequently
 - Known as the **Dispatcher**

Two Suspend States

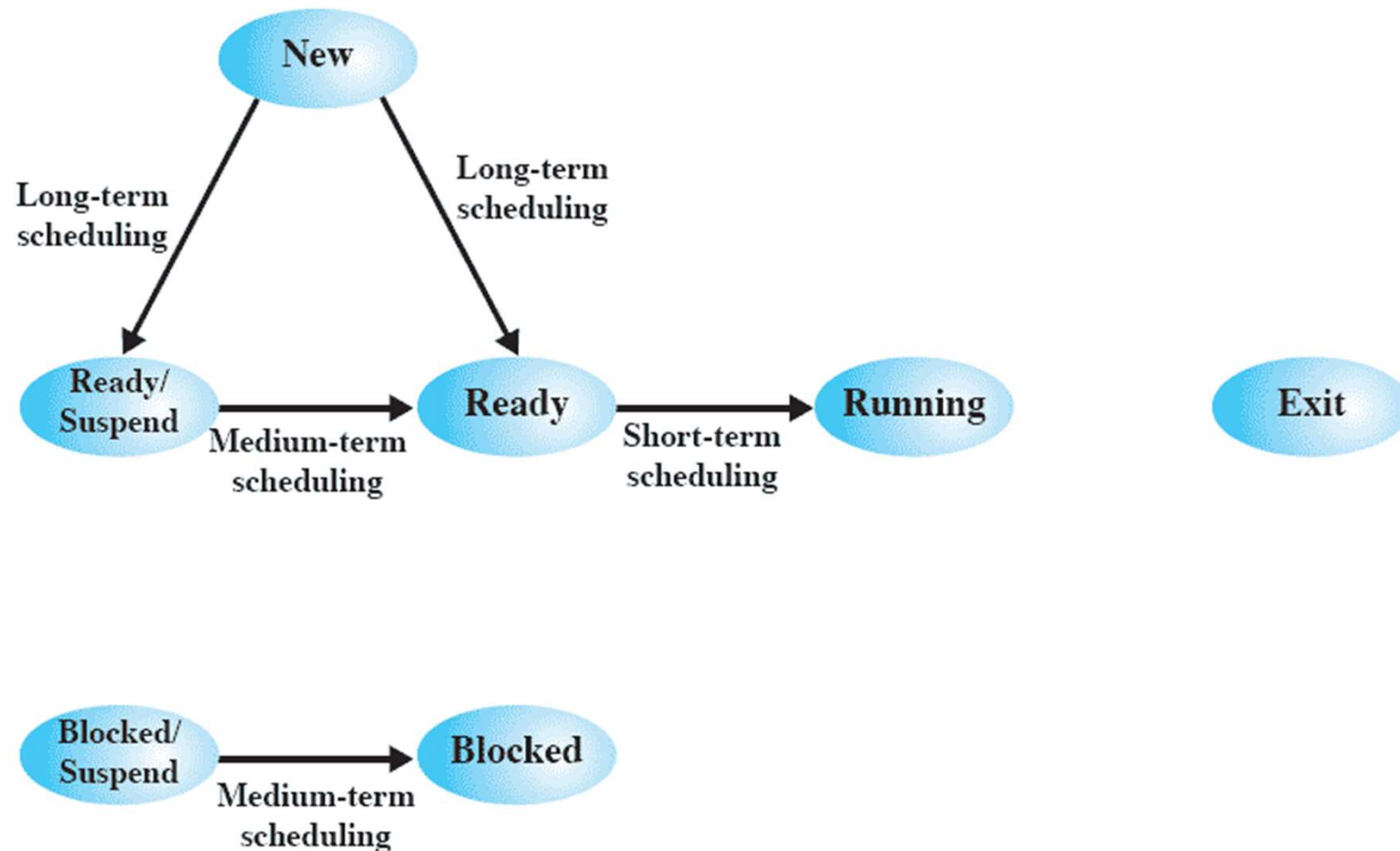
Remember this diagram



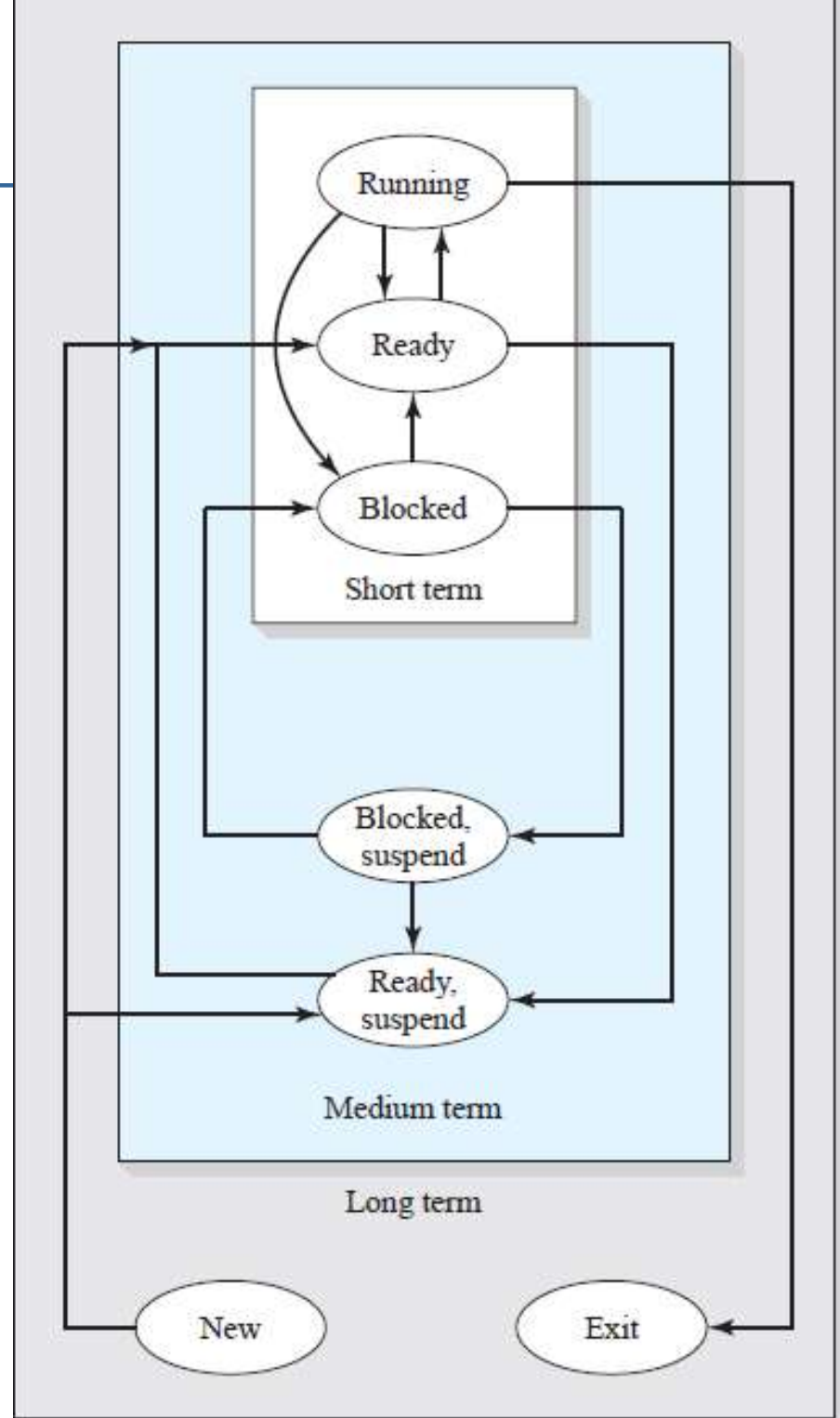
(b) With Two Suspend States

Where do the long/medium/short-term schedulers intervene?

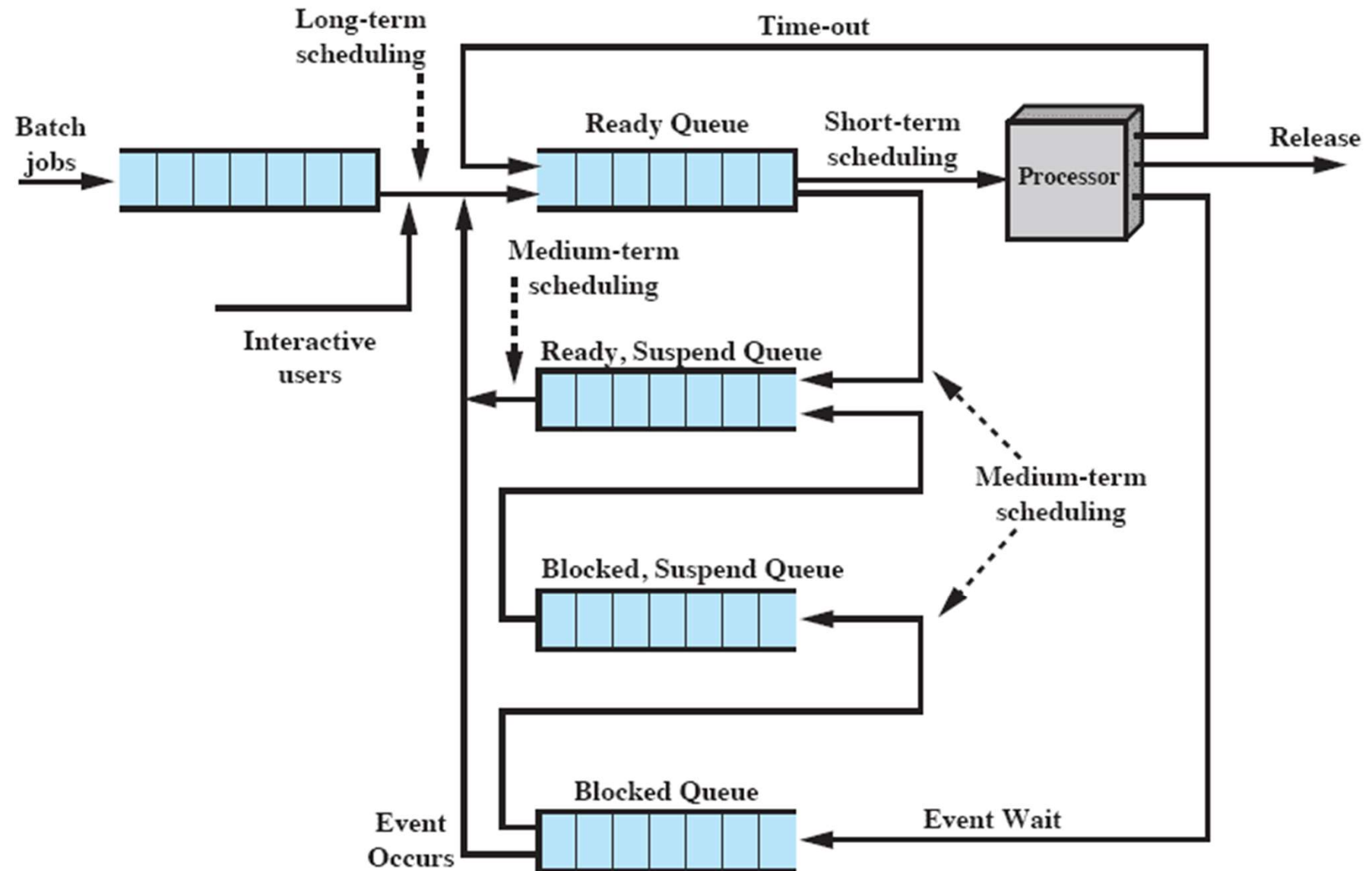
Scheduling and Process State Transitions



Levels of Scheduling

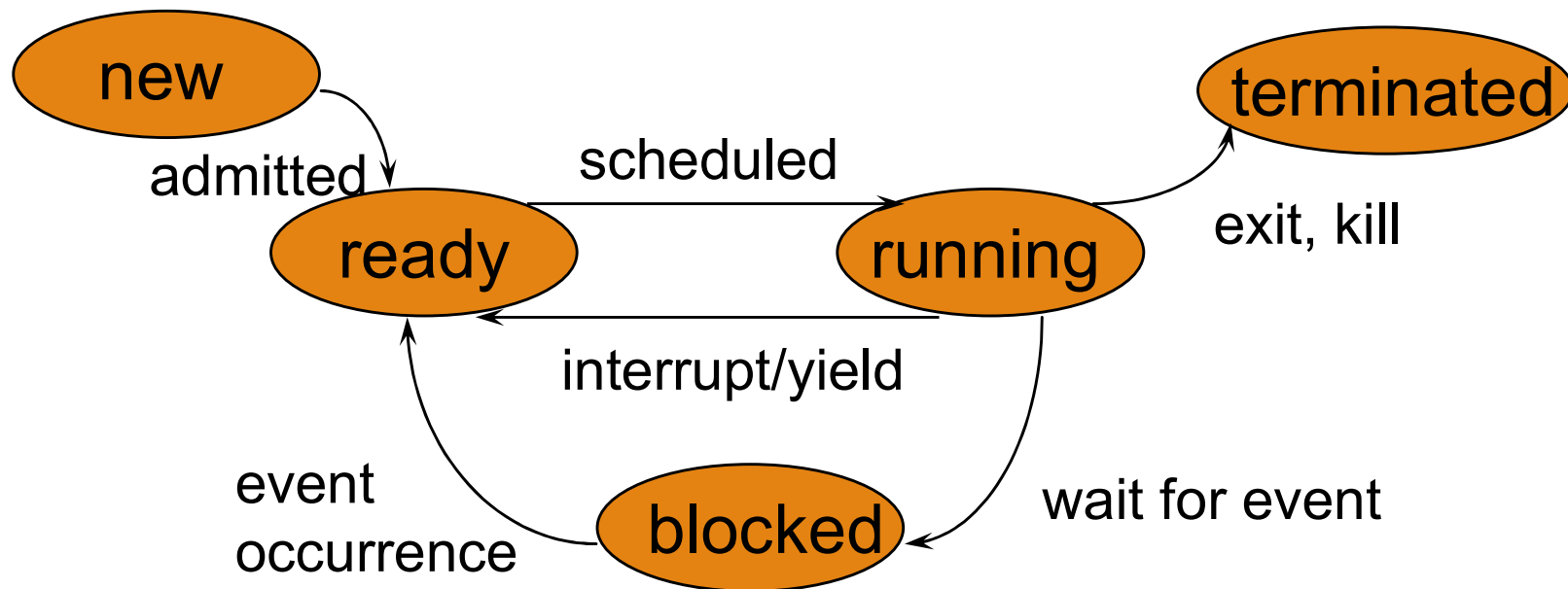


Queuing Diagram for Scheduling



When is short-term scheduling invoked?

- Scheduling absolutely required when:
 - A process exists
 - A process becomes blocked (on I/O, etc)
- Scheduling may be required when:
 - New process created
 - I/O interrupt
 - Clock interrupt



Preemptive vs. Non-Preemptive Algorithms

- **Non-preemptive**: A process runs until it voluntarily gives up the CPU
 - a) Terminates
 - b) Blocks itself waiting for I/O or service
 - ▶ used when suspending a process is impossible or very expensive
- **Preemptive**: A Running Process can be **interrupted** and moved to the Ready state, then **resumed** later
 - ▶ Preemption may occur when
 - new process arrives
 - on an interrupt that placed a blocked process in Ready
 - periodically, based on clock interrupt
 - Most OS use preemptive CPU scheduling, implemented via timer interrupt
 - -- incurs greater overhead (**context switch**)
 - ++ Allows for better service since it prevents monopolization of CPU

Different Needs

- **Batch**: no terminal users
 - payroll, inventory, interest calculation, etc.
 - preemption or non-preemption is acceptable
- **Interactive**: lots-o-I/O from users
 - Preemption is needed to avoid one process from monopolizing CPU and denying services to others
- **Real-time**: process must run and complete on time
 - Typically real-time only runs processes specific to the application at hand

Observations

- Processes are dependent on I/O

 - dependency level varies

- Processes cycle

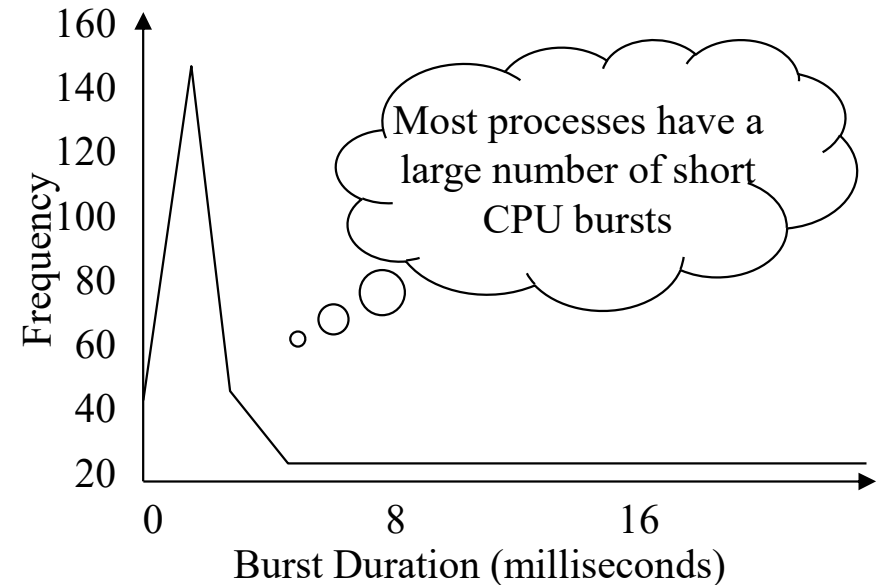
 - compute

 - wait for I/O

- Process execution is characterized by

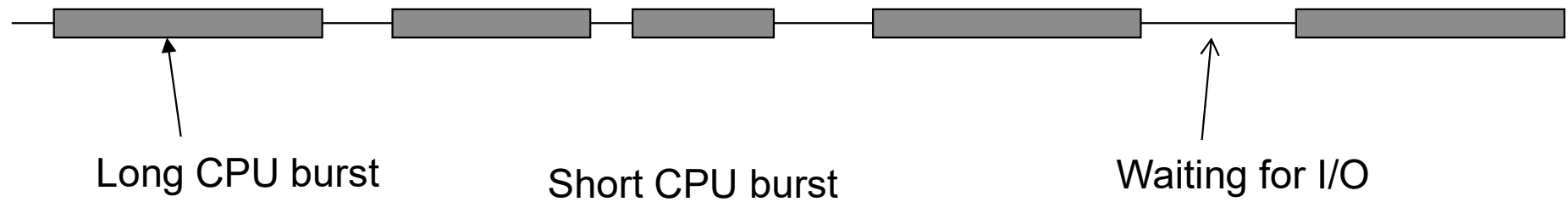
 - length of CPU burst

 - number of bursts

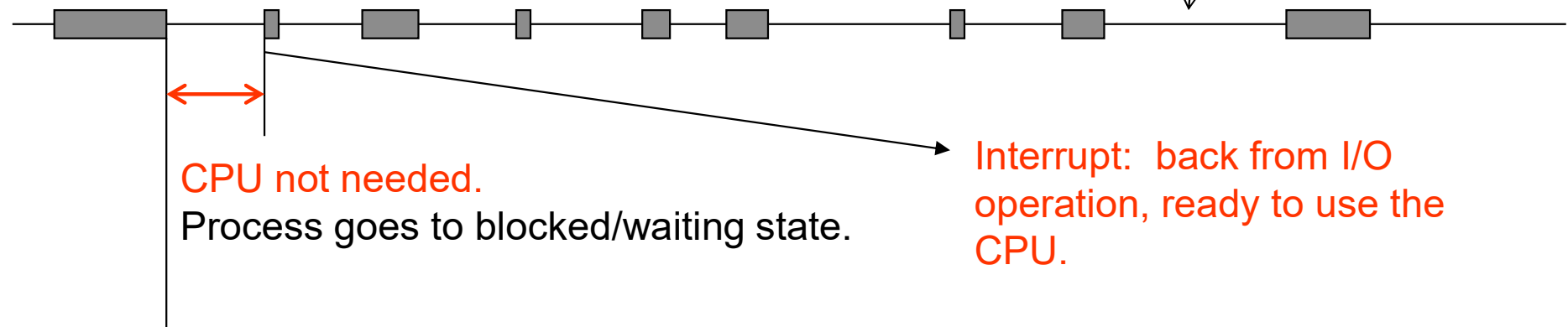


CPU and I/O Bursts

CPU-bound



I/O-bound



As CPU gets faster, processes tend to get more I/O-bound

Response Time

- User productivity tends to improve with a more rapid response time
 - Especially true for expert users
 - Becomes very noticeable as response time drops below 1 second
- User time or “think time” – Time user spends thinking about the response.
- System time – Time system takes to generate its response.
 - Short response times are important
 - User response time tends to decrease as system response time decreases
 - If the system gets too slow to a user, they may slow down or abort the operation
- Turnaround time (TAT) – total time that an item spends in the system (waiting time + service time)

Response Times (continued...)

15 seconds or greater	•Rules out conversational interaction. •Most users will not wait this long. •If it cannot be avoided, allow the user to continue on to something else (like foreground/background threads).
4 to 15 seconds	•Generally user loses track of the current operation in short-term memory •Ok after end of a major closure
2 to 4 seconds	•Inhibits user concentration •Bothers a user who is focused on the job •Ok after end of a minor closure
1 to 2 seconds	•Important when information has to be remembered over several responses •Important limit for terminal activities
Less than 1 second	•For keeping user's attention for thought-intensive activities •Example: graphics
Less than 1/10 second	•Response to key presses or mouse clicks

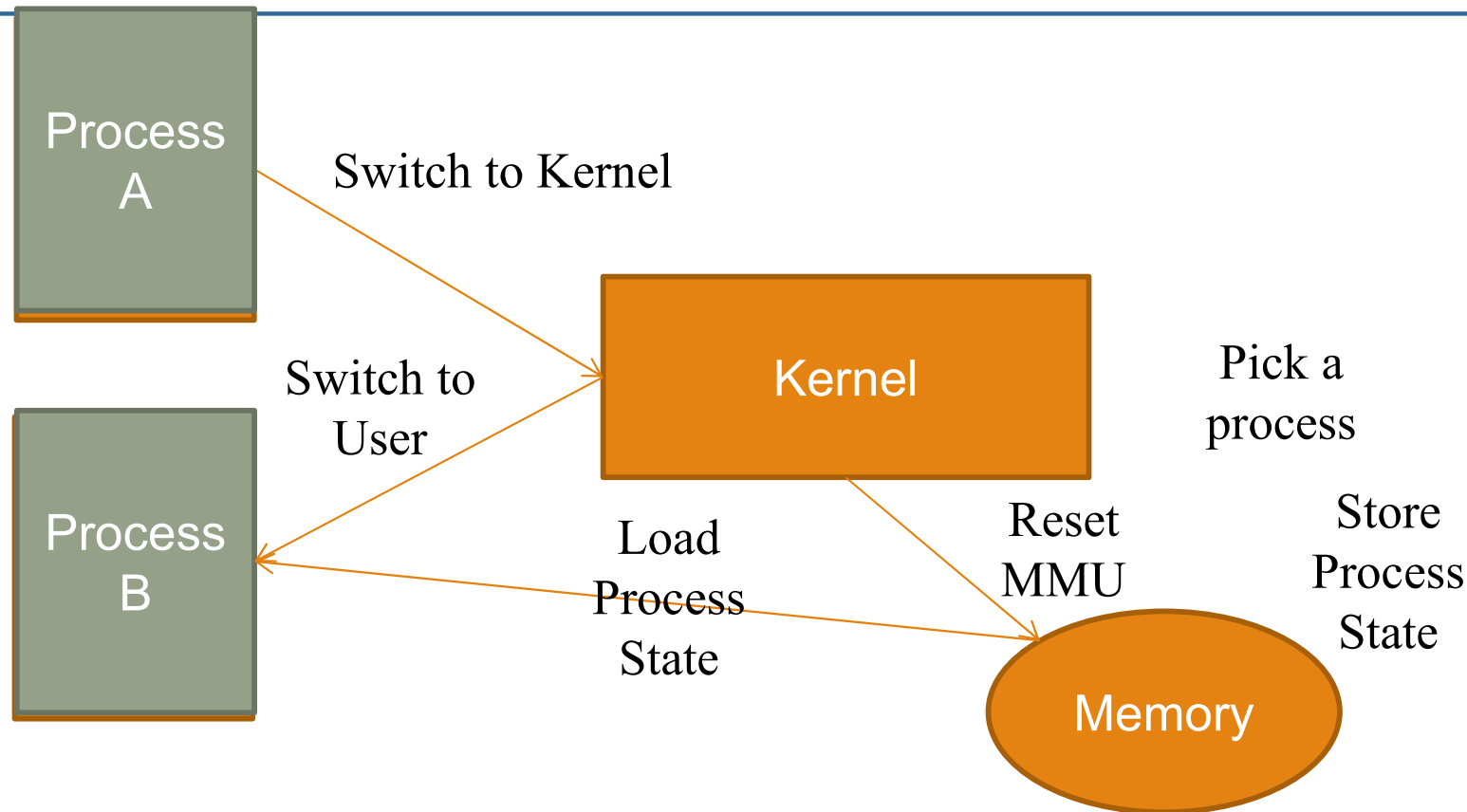
Aim of Scheduling

- Assign processes to be executed by the processor(s) needs to meet system objectives, such as:
 - minimize **response time**
 - maximize **throughput** (jobs/minute)
 - maximize processor efficiency (**utilization**)
 - support **multiprogramming**
- The scheduling function should
 - Share time **fairly** among processes
 - Prevent **starvation** of a process
 - Have **low overhead**
 - **Prioritise** processes when necessary (e.g. real time deadlines)

Evaluation Criteria for Short-Term Scheduling Policies

- User-oriented -- performance related
 - **Turnaround time**: time from submission to completion (batch jobs)
 - **Response Time**: time between submission of request and the first output. (interactive users)
- User-oriented -- other
 - **Predictability**: runs the same way (Time & cost) regardless of the system load
- System-oriented -- Performance related
 - **Maximize Throughput**: rate at which processes are completed (jobs processed / unit time)
 - **Processor utilization**: percentage of time that the processor is busy
- System-oriented -- other
 - **Fairness** share time fairly among processes
 - **No Starvation**; No process should suffer starvation
 - **Enforcing priorities** Favor higher priority processes
 - **Balancing resources** Keep system resources busy (mix I/O+CPU bound)
 - Have **low overhead**

Switching is Expensive



- Overhead

- Direct Cost: time to actually switch
- Indirect Cost: performance hit from memory (invalidate & reload cache, swapped out pages, etc.)

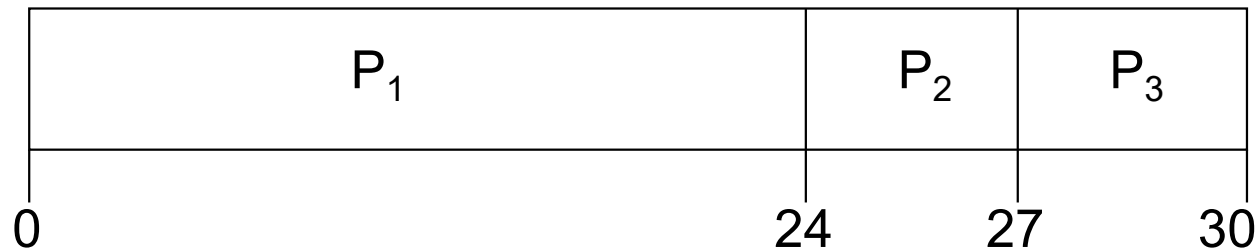
→ doing too many process switches can chew up CPU time

Scheduling Algorithms

1. First Come First Served (**FCFS**)
2. Round Robin (**RR**) – time slicing.
3. Shortest Process Next (**SPN**)
4. Shortest Remaining Time (**SRT**)
5. Highest Response Ratio Next (**HRRN**)
6. Feedback (**FB**)

FCFS Scheduling

- First Come, First Served.
- Assume non-preemptive version
- Example: Jobs are P_1 (duration: 24 units); P_2 (3); and P_3 (3), arriving in this order at time 0



- Average waiting time: $(0 + 24 + 27)/3 = 17$

FCFS: Continued

What if execution order is P_2 , P_3 , P_1 ?



- Average waiting time: $(6 + 0 + 3)/3 = 3$
- **Much** better!
- *Convoy effect*: many short jobs stuck behind one long job
- ++ Easy to implement: queue with single linked list; pick front of queue, service, place at end of queue
- ++ Fair
- -- penalize short jobs in the presence of long jobs

Process	Arrival Time	Service Time (T_s)	Start Time	Finish Time	Turnaround Time (T_r)	T_r/T_s
W	0	1	0	1	1	1
X	1	100	1	101	100	1
Y	2	1	101	102	100	100
Z	3	100	102	202	199	1.99
Mean					100	26

- **turnaround time** = total time that the job spends in the system
- **normalized turnaround time** = ratio of turnaround time to service time - indicates the relative delay experienced
- The normalized turnaround time for process Y is worst: total time in the system is 100 times the required processing time.
- long processes do not suffer: Process Z has a turnaround time that is almost double that of Y, but its normalized residence time is under 2.0

Process Scheduling Example

- Example set of processes, consider each a batch job

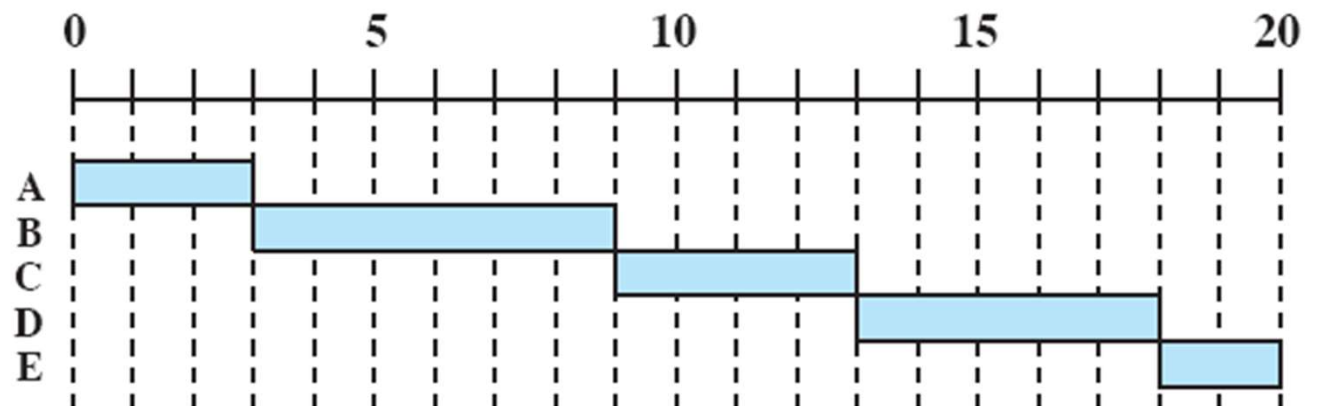
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

– Service time represents total execution time

First-Come-First-Served FCFS

- ❑ Each process joins the Ready queue
- ❑ Non-preemptive: a process relinquish the CPU if it blocks for I/O or exits
- ❑ The **oldest process** in the **Ready queue** is selected to run next
- ❑ **Disadvantage:**
 - ❑ A short process have to wait a long time before it can execute
 - ❑ Favors CPU-bound processes : I/O processes have to wait until CPU-bound process completes
- ❑ Often combined with a priority scheme to provide an effective scheduler

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

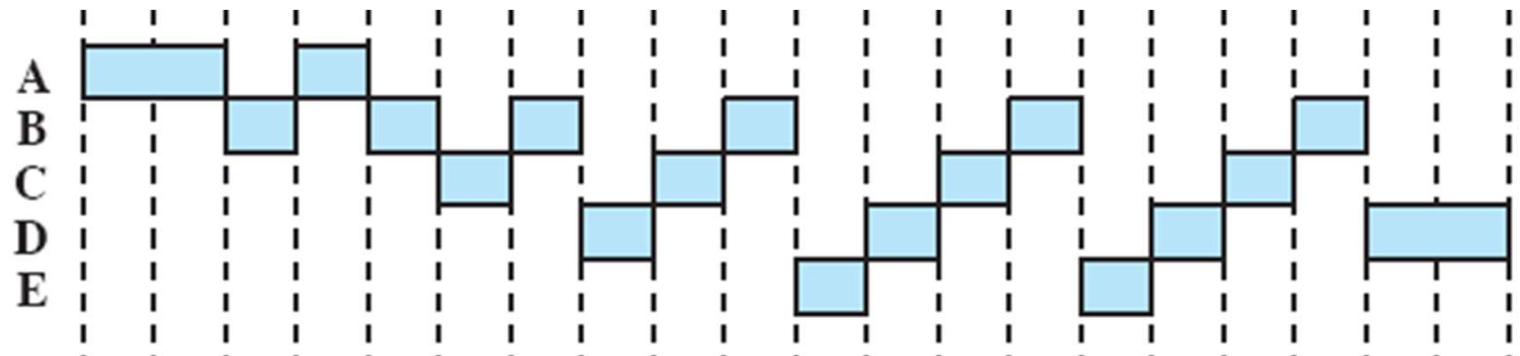


Round Robin

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

- Each process gets a small unit of CPU time (**time quantum/ time slice**), usually 20-50 milliseconds.
- **Preemption** based on a clock (**time slicing**)
- Clock interrupt is generated at periodic intervals
- When an interrupt occurs, the currently running process is placed in the Ready queue and next ready job is selected based on FCFS to run

Round-Robin
(RR), $q = 1$



Effect of Length Of Quantum

□ If too short

++ short processes will move quickly

-- too many overhead (context switching), reduce CPU efficiency

▶ If quantum is 4ms, Context Switch cost is 1ms

▶ For every 4ms of work, kernel does 1ms

→ 20% overhead

→ **avoid short quantum**

□ If too long

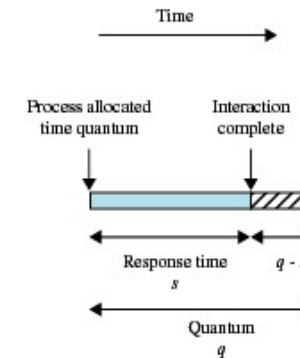
□ Suppose time quantum is 100ms

→ Now overhead is 1%

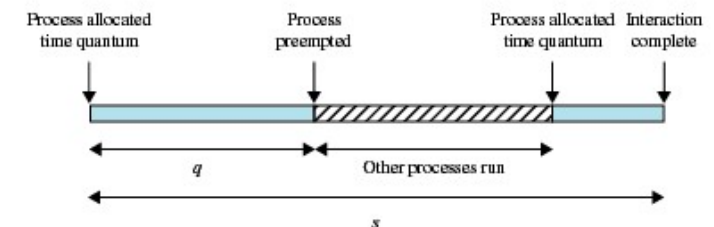
-- Poor performance to short interactive requests

□ **Guide:** quantum should be slightly greater than the time required for a typical interaction

□ So apps block before quantum expires, context switching is done only once



(a) Time quantum greater than typical interaction



(b) Time quantum less than typical interaction

Figure 9.6 Effect of Size of Preemption Time Quan

'Virtual Round Robin'

- RR unfair to I/O-bound processes
 - I/O bound give up their time slice while CPU bound use their complete time quantum.
 - I/O bound have to wait for CPU-bound to relinquish CPU
 - Poor performance for I/O-bound processes
 - inefficient use of I/O devices
 - increase in the variance of response time
- Could be improved using a Virtual Round Robin (**VRR**) scheduling scheme
 - Add an **auxiliary queue** where processes are placed after released from IO block
 - At dispatch, processes in auxiliary queue get preference over the main queue.

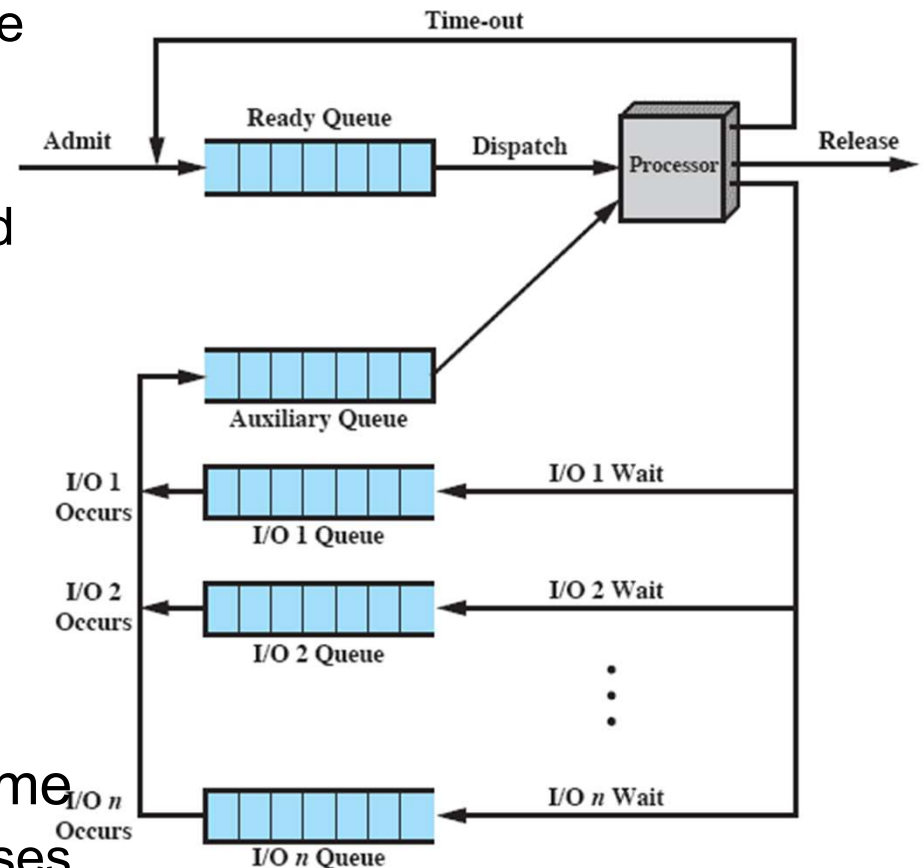


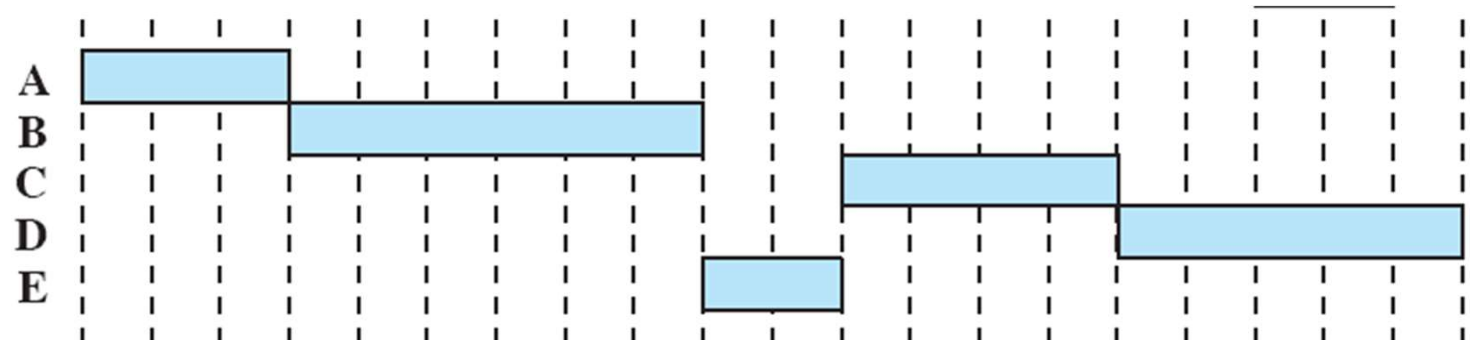
Figure 9.7 Queuing Diagram for Virtual Round-Robin Scheduler

Shortest Process Next (SPN)

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

- Also known as **Shortest Job First**
- **Non-preemptive**
 - Assume each job has a *known* execution time: provided or estimated
- Process with shortest **expected** processing time is selected next
- Short process jumps ahead of longer processes
- Performance
 - Improvement in response time
 - Variability of response time increases, specialty for longer processes, thus predictability of longer processes is reduced

Shortest Process
Next (SPN)



Estimating Execution Time

- SPN needs to **estimate** the required processing time of a process
 - For **batch jobs**, require a programmer's estimate.
 - If estimate is substantially off, system may abort job.
 - In an **interactive environment**, OS may keep a running average of each “burst” for each process.
 - Estimate based on **past behavior**

$$S_{n+1} = \frac{1}{n} \sum_{i=1}^n T_i$$

- T_i = actual CPU execution time for the i^{th} instance of this process
- S_i = predicted value for the i^{th} instance
- S_1 = predicted value for first instance; not calculated
- But this formula gives equal weight to each observation, while it is desirable to give more weight to more recent observations

Estimating Execution Time

- A common technique for predicting a future value on the basis of a time series of past values is **exponential averaging**

$$S_{n+1} = \alpha T_n + (1 - \alpha)S_n$$

- Where α is a constant weighting factor, $0 < \alpha < 1$, determining the relative weight given to recent observations relative to older ones
- Expanding the above equation

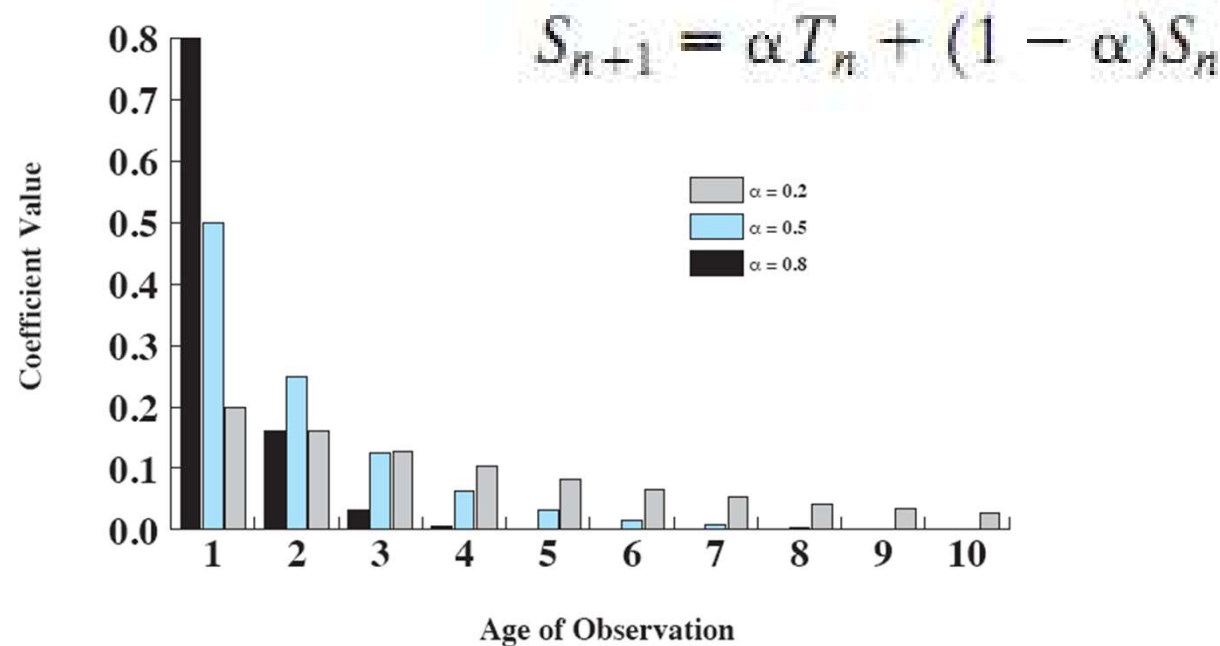
$$S_{n+1} = \alpha T_n + (1 - \alpha)\alpha T_{n-1} + \dots + (1 - \alpha)^i \alpha T_{n-i} + \dots + (1 - \alpha)^n S_1$$

- For $\alpha=0.8$

$$S_{n+1} = 0.8T_n + 0.16T_{n-1} + 0.032 T_{n-2} + 0.0064 T_{n-3} + \dots$$

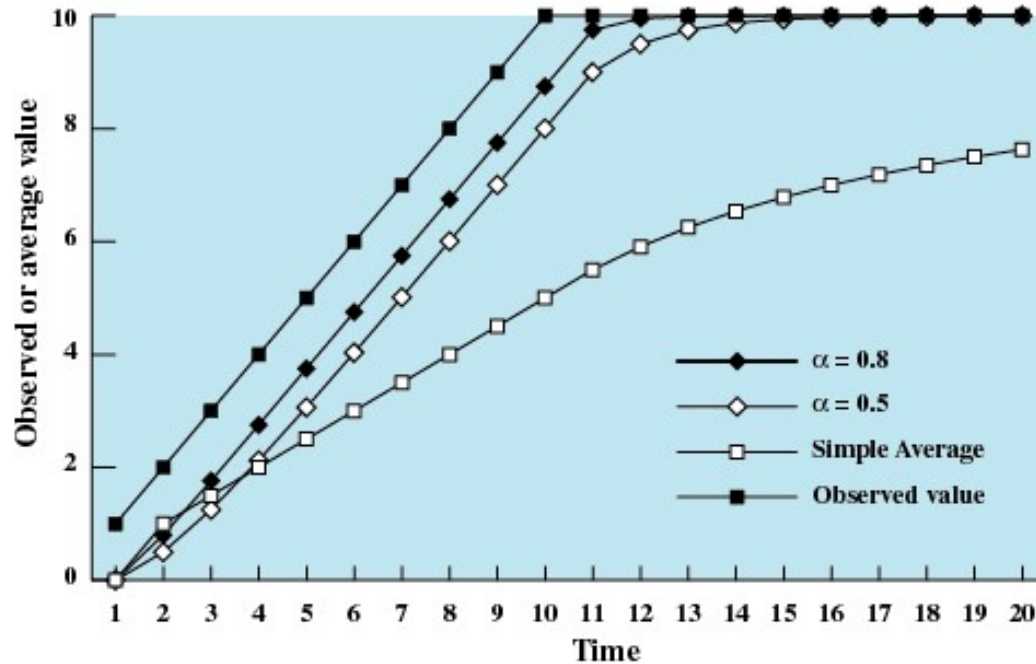
- The older the observation, the less it is counted in to the average
- α decide to have the estimation process forget old runs quickly, or remember them for a long time
- This technique is sometimes called **aging**.

Exponential Smoothing Coefficients

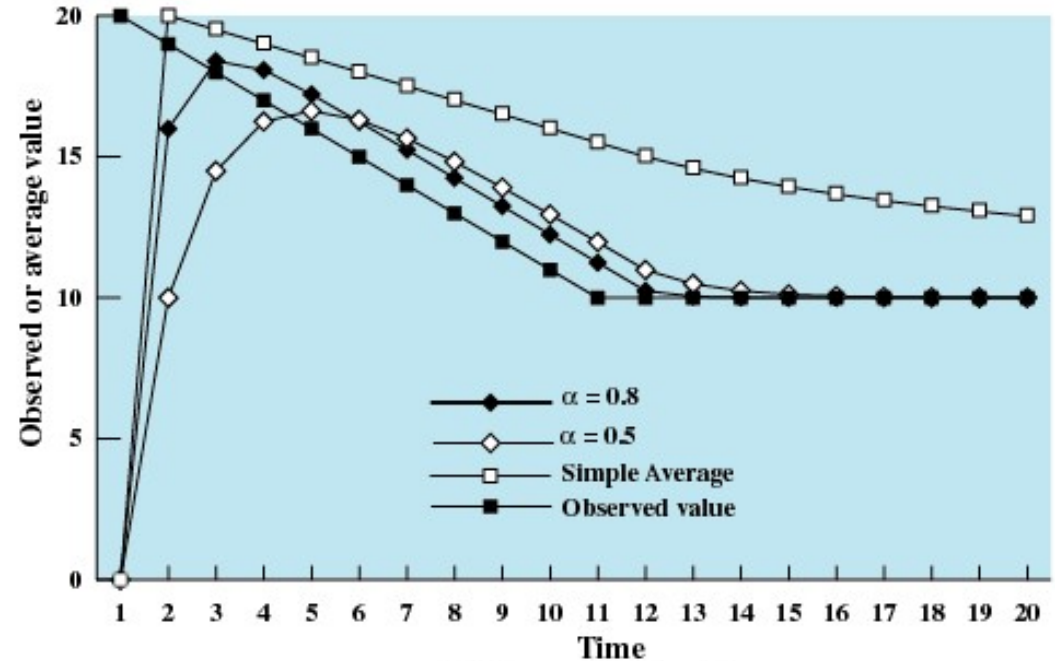


- The larger α , the greater the weight given to the more recent observations
 - $\alpha = 0.8$, most of the weight is given to the four most recent observations
 - $\alpha = 0.2$, the averaging is spread out over the eight or so most recent observations
- The advantage of using a value of α close to 1 is that the average will quickly reflect a rapid change in the observed quantity
- The disadvantage is that if there is a brief surge in the value of the observed quantity and it then settles back to some average value, the use of a large value of α will result in jerky changes in the average

Use Of Exponential Averaging



(a) Increasing function

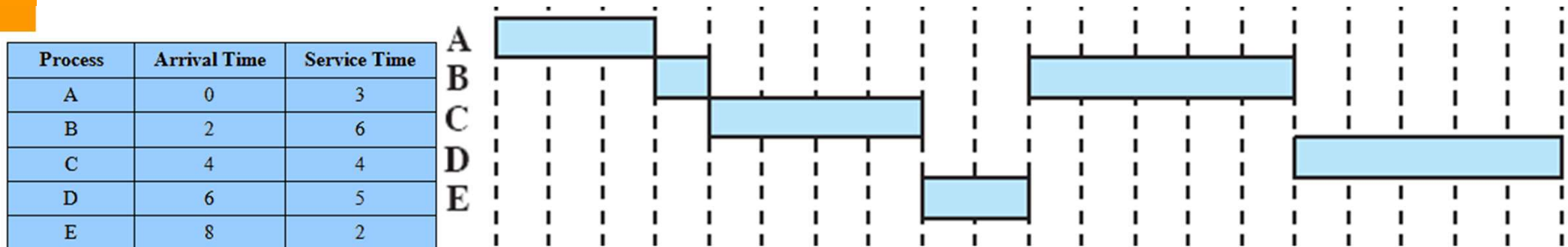


(b) Decreasing function

- The exponential averaging tracks changes in process behaviour faster than does simple averaging
- the larger value of α results in a more rapid reaction to the change in the observed value

Shortest Remaining Time (SRT)

- **Preemptive** version of Shortest Process Next (**SPN**)
 - If a new process arrives with estimated processing times less than the remaining time of current executing process → preempt running process
- Must **estimate processing time** and choose the shortest
- Achieves better turnaround time vs. SPN, because a short job is given immediate preference to a running longer job
- SRT does not bias in favor of long processes (as does FCFS)
- Unlike RR, no additional interrupts are generated reducing overhead.
- But elapsed service times must be recorded, contributing to overhead
- Risk of starvation for long jobs



Highest Response Ratio Next (HRRN)

- An important performance criteria to **minimize** is:

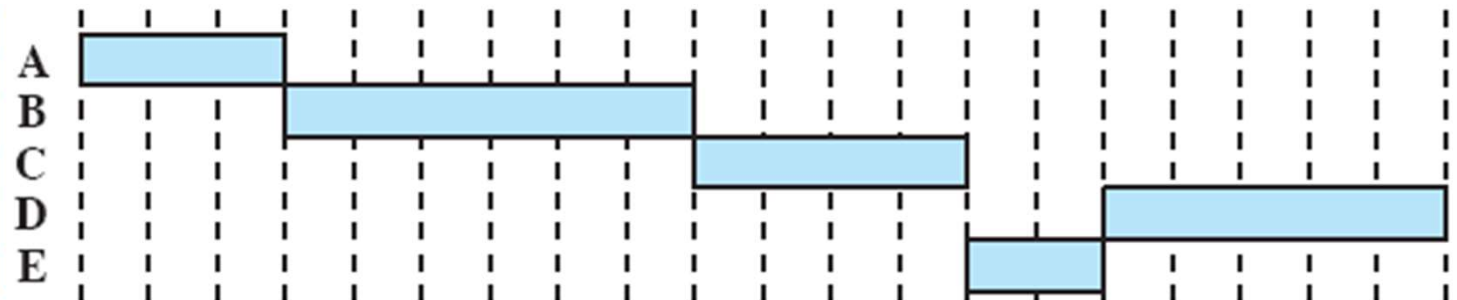
$$\text{Normalized TurnAround Time} = \text{TAT} / \text{ServiceTime}$$

- We can approximate it based on past history. So consider

$$\text{Ratio} = \frac{\text{time spent waiting} + \text{expected service time}}{\text{expected service time}}$$

- Choose next process with the **highest** response ratio R
- Attractive because it accounts for the **age** of a process.
- While **shorter jobs** are **favored** (a smaller denominator yields a larger ratio), **aging** without service **increases** the ratio so that a **longer process** will eventually get past competing shorter jobs.

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



Priorities

- Processes are assigned priorities
- Scheduler will choose a *Ready* process of higher priority
- Have **multiple ready queues** to represent each level of priority
 - Scheduler starts with RQ0
 - If RQ0 is empty, then RQ1 is examined
- Lower-priority may suffer **starvation**
 - Allow a process to change its priority based on its **age** or execution history
- Note: All scheduling policies are priority scheduling!
- Question:** How to assign priorities?

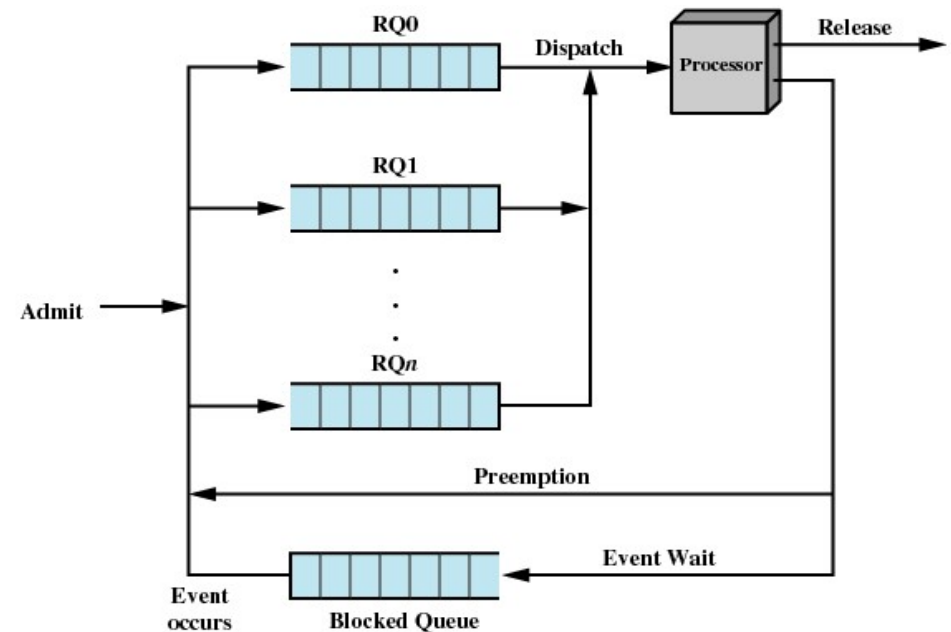
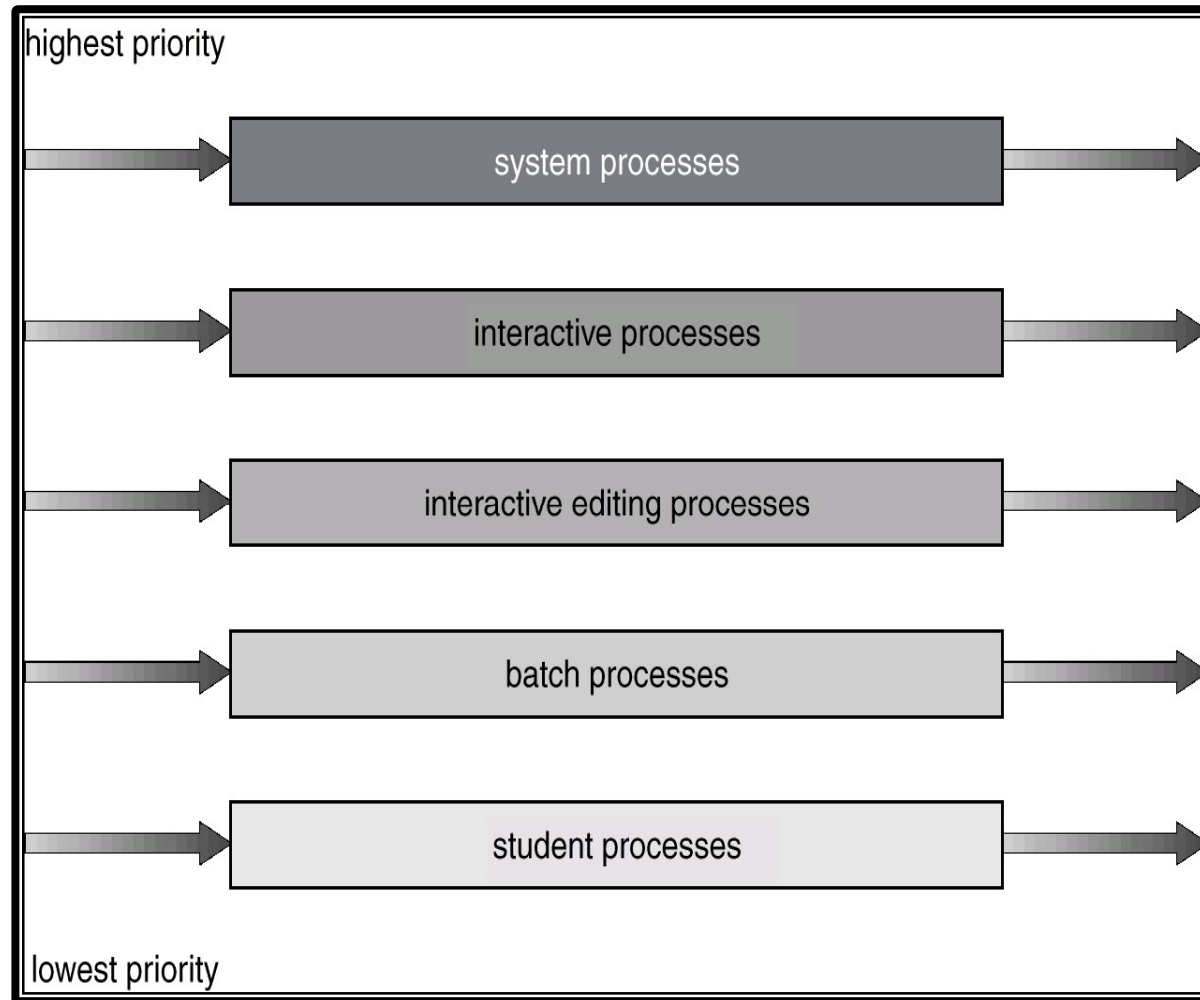


Figure 9.4 Priority Queuing

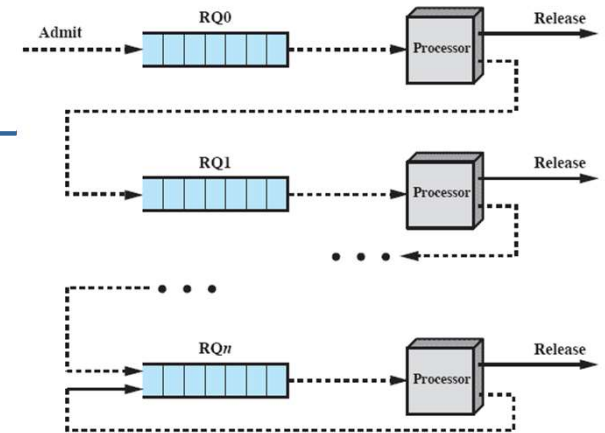
Example for Priorities



Static priorities can lead to starvation!

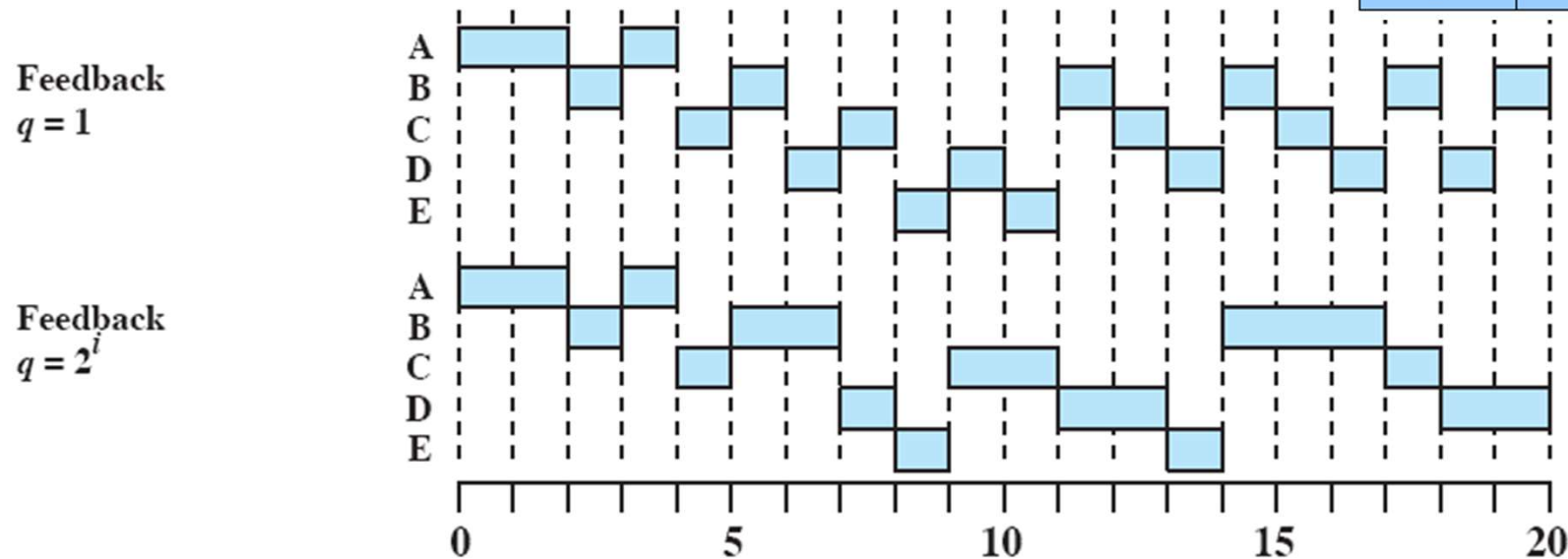
Feedback Scheduling

- ❑ If it is hard to predict the **remaining time** a process then SPN, SRT, and HRRN cannot be used.
- ❑ give preference for shorter jobs by **penalizing** jobs that have been **running longer**.
 - ▶ ie. if we cannot focus on time remaining, focus on time spent in execution so far.
- ❑ Feedback scheduling is done on a **preemptive** basis with a **dynamic priority** mechanism.
- ❑ A process is placed in RQ0, after 1st preemption, it is placed in RQ1
- ❑ A process is **demoted** to the next **lower-priority queue** each time it is preempted and returns to the ready queue
- ❑ Within each queue, FCFS is used except once in the lowest-priority queue, a process cannot go lower and is treated in a RR fashion.
- ❑ With this strategy
 - ❑ A short process will complete quickly
 - ❑ a longer process will gradually drift downward
 - ❑ Newer, shorter processes are favored over older and longer
 - ❑ Turnaround time for longer processes can be intolerable.
 - ❑ Starvation may occur if new jobs enter the system frequently



Feedback Performance

Process	Arrival Time	Service Time
A	0	3
B	2	2
C	4	2
D	6	2
E	8	2

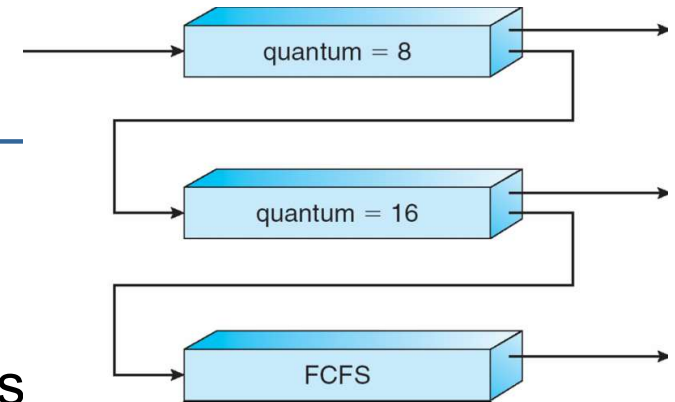


- ❑ **Variation 1:** perform preemption in the same fashion as RR: at periodic intervals. Example $q=1$, behaviour similar to RR
- ❑ **Variation 2:** vary the preemption times according to the queue:
 - ▶ A process scheduled from RQ0 is allowed to execute for one time unit ,then is preempted;
 - ▶ a process scheduled from RQ1 is allowed to execute two time units, and so on.
 - ▶ Still a longer process may still suffer
- ❑ **Variation 3:** promote a process to a higher-priority queue after it spends a certain amount of time waiting for service in its current queue

Multilevel Feedback Queue

- A process can move between the various queues; **aging** can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:
 - number of queues
 - scheduling algorithms for each queue
 - method used to determine when to upgrade a process
 - method used to determine when to demote a process
 - method used to determine which queue a process will enter when that process needs service

Example of MFQ



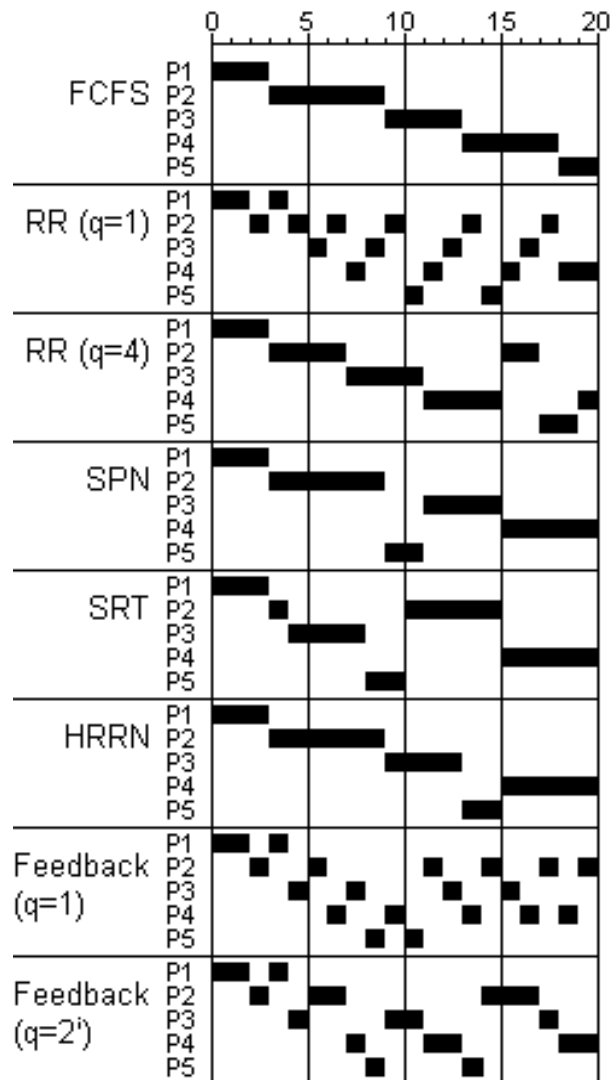
□ Three queues:

- Q_0 – RR with time quantum 8 milliseconds
- Q_1 – RR time quantum 16 milliseconds
- Q_2 – FCFS

□ Scheduling

- A new job enters queue Q_0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q_1 .
- At Q_1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2 .

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



	A	B	C	D	E	Mean
Arrival	0	2	4	6	8	
Service Time	3	6	4	5	2	
FCFS						
Finish	3	9	13	18	20	
Turnaround	3	7	9	12	12	8.60
Tr / Ts	1.00	1.17	2.25	2.40	6.00	2.56
RR (q=1)						
Finish	4	18	17	20	15	
Turnaround	4	16	13	14	7	10.80
Tr / Ts	1.33	2.67	3.25	2.80	3.50	2.71
RR (q=4)						
Finish	3	17	11	20	19	
Turnaround	3	15	7	14	11	10.00
Tr / Ts	1.00	2.50	1.75	2.80	5.50	2.71
SPN						
Finish	3	9	15	20	11	
Turnaround	3	7	11	14	3	7.60
Tr / Ts	1.00	1.17	2.75	2.80	1.50	1.84
SRT						
Finish	3	15	8	20	10	
Turnaround	3	13	4	14	2	7.20
Tr / Ts	1.00	2.17	1.00	2.80	1.00	1.59
HRRN						
Finish	3	9	13	20	15	
Turnaround	3	7	9	14	7	8.00
Tr / Ts	1.00	1.17	2.25	2.80	3.50	2.14
Feedback (q=1)						
Finish	4	20	16	19	11	
Turnaround	4	18	12	13	3	10.00
Tr / Ts	1.33	3.00	3.00	2.60	1.50	2.29
Feedback (q=2⁽ⁱ⁻¹⁾)						
Finish	4	17	18	20	14	
Turnaround	4	15	14	14	6	10.60
Tr / Ts	1.33	2.50	3.50	2.80	3.00	2.63

Algorithms

□ First Come First Served

- Processes queued in order of arrival
- Runs until finished or blocks on I/O
- Tends to penalize short processes (have to wait for long processes)
- Favors CPU-bound processes (I/O process quickly block)

□ Round Robin

- FCFS with preemption
- Size of ticks affects performance
- Favors processor-bound processes
- Virtual Round Robin
 - ▶ Second queue for formerly blocked processes – given priority
 - ▶ At end of time slice, add to end of standard queue

Algorithms (continued...)

□ Shortest Process Next

- Select process with shortest expected running time (non-preemptive)
- Difficult to estimate required time (keep history)
- Tends to be less predictable
- Can starve long processes
- Short processes may still wait if a long process has just started

□ Shortest Remaining Time

- Preemptive version of Shortest Process Next
- May switch processes when a new process arrives
- Still may starve long processes

Algorithms (continued...)

□ Highest Response Ratio Next

- Non-preemptive, tries to get best average normalized turnaround time
 - Depends on Response Ratio
 - ▶ W = time spent waiting
 - ▶ S = expected service time
 - Select process with highest R
- $$R = (W + S) / S$$

□ Feedback

- Starts in high-priority queue, moves down in priority as it executes
- Lower-priority queues often given longer time slices
- Can starve long processes
 - ▶ Move up processes if they wait too long

Table 9.3 Characteristics of Various Scheduling Policies

	Selection Function	Decision Mode	Throughput	Response Time	Overhead	Effect on Processes	Starvation
FCFS	$\max[w]$	Nonpreemptive	Not emphasized	May be high, especially if there is a large variance in process execution times	Minimum	Penalizes short processes; penalizes I/O bound processes	No
Round Robin	constant	Preemptive (at time quantum)	May be low if quantum is too small	Provides good response time for short processes	Minimum	Fair treatment	No
SPN	$\min[s]$	Nonpreemptive	High	Provides good response time for short processes	Can be high	Penalizes long processes	Possible
SRT	$\min[s - e]$	Preemptive (at arrival)	High	Provides good response time	Can be high	Penalizes long processes	Possible
HRRN	$\max\left(\frac{w + s}{s}\right)$	Nonpreemptive	High	Provides good response time	Can be high	Good balance	No
Feedback	(see text)	Preemptive (at time quantum)	Not emphasized	Not emphasized	Can be high	May favor I/O bound processes	Possible

w = time spent waiting

e = time spent in execution so far

s = total service time required by the process, including e

Fair-Share Scheduling (FSS)

- ❑ User's application runs as a collection of processes (threads)
- ❑ User is concerned about the performance of the **application**
- ❑ Need to make scheduling decisions based on **groups of process** belonging to an application
- ❑ Each **user** is **assigned** a **share** of the processor
- ❑ Concept is extended to groups of users
- ❑ Objective is to monitor usage to give fewer resources to users who have had more than their fair share and more to those who have had less than their fair share
- ❑ FSS is based on
 - ❑ The execution history of each process
 - ❑ The execution history of a related group of processes

Fair-Share Scheduling

- FSS is based on dynamic priority, considering
 - Base priority of the process
 - Recent processor usage of the process
 - Recent processor usage of the group the process belongs to

$$\begin{aligned}CPU_j(i) &= \frac{CPU_j(i-1)}{2} \\GCPU_k(i) &= \frac{GCPU_k(i-1)}{2} \\P_j(i) &= Base_j + \frac{CPU_j(i)}{2} + \frac{GCPU_k(i)}{4 \times W_k}\end{aligned}$$

where

$CPU_j(i)$ = measure of processor utilization by process j through interval i
 $GCPU_k(i)$ = measure of processor utilization of group k through interval i
 $P_j(i)$ = priority of process j at beginning of interval i ; lower values equal higher priorities
 $Base_j$ = base priority of process j
 W_k = weighting assigned to group k , with the constraint that $0 < W_k \leq 1$ and $\sum_k W_k = 1$

- The higher the numerical value, the lower the priority
- The priority of a process drops:
 - ▶ As the process uses the CPU, and
 - ▶ As the group the process belongs to uses the CPU

Fair-Share Scheduler

- A is in one group, B and C in another group, each group has a weighting of 0.5
- A,B,C are CPU-bound → usually ready to run
- All processes have a base priority of 60
- Processor utilization is measured as follows:
 - CPU is interrupted 60 times per second;
 - during each interrupt, the processor usage field of the currently running process is incremented, as is the corresponding group processor field.
 - Once per second, priorities are recalculated⁴
- In the figure, process A is scheduled first.
 - At the end of one second, it is pre-empted⁵
 - Processes B and C now have the higher priority, and process B is scheduled
- At the end of the second time unit, process A has the highest priority
- the pattern repeats: A, B, A, C, A, B, and so on. Figure 10.10

Time	Process A			Process B			Process C		
	Priority	Process CPU count	Group CPU count	Priority	Process CPU count	Group CPU count	Priority	Process CPU count	Group CPU count
0	60	0	0	60	0	0	60	0	0
1		1	1						
		2	2						
		•	•						
		•	•						
		60	60						
2	90	30	30	60	0	0	60	0	0
3					1	1			1
					2	2			2
					•	•			•
					•	•			•
					60	60			60
4	74	15	15	90	30	30	75	0	30
5		16	16						
		17	17						
		•	•						
		•	•						
		75	75						
6	96	37	37	74	15	15	67	0	15
7					16	16		1	16
					17	17		2	17
					•	•		•	•
					•	•		•	•
					75	75		60	75
8	78	18	18	81	7	37	93	30	37
9		19	19						
		20	20						
		•	•						
		•	•						
		78	78						
10	98	39	39	70	3	18	76	15	18

Group 1 Group 2

Colored rectangle represents executing process

• **Figure 9.16** Example of Fair Share Scheduler—Three Processes, Two Gr

Scheduling Algorithm Evaluation

- How do we pick an algorithm?
 - Many algorithms
 - Many parameters
- Maximize or Minimize some criteria.
 - Utilization, throughput, etc.
- What do we base our choice on?
 - A single example?

Deterministic Modeling

- Given a predetermined workload, simulate
 - run the workload through different schedulers
 - calculate statistics

Process	Burst Time
P1	10
P2	29
P3	3
P4	7
P5	12

Algorithm Evaluation: Deterministic Modeling

FCFS	P1	P2	P3	P4	P5
------	----	----	----	----	----

$$\text{Ave Wait} = (0 + 10 + 39 + 42 + 49)/5 = 28$$

SPN	P3	P4	P1	P5	P2
-----	----	----	----	----	----

$$\text{Ave Wait} = (10 + 32 + 0 + 3 + 20)/5 = 13$$

RR/10	P1	P2	P3	P4	P5	P2	P5	P2
-------	----	----	----	----	----	----	----	----

$$\text{Ave Wait} = (0 + 32 + 20 + 23 + 40)/5 = 23$$

What about context switch overhead?

Advantages

Simple

Fast

Exact Results

Disadvantages

too specific

too much exact knowledge is required

tied to example data

Queuing Theory

- Using statistics, we can determine the distribution of CPU and I/O bursts.
 - Probability distribution function
- The result is a mathematical formula which describes the probability of a particular burst
- Mathematics can then tell us performance
- A computer system can be described as a network of servers
 - each server has a queue of waiting processes
- Simply add an imaginary server to each queue.
- Now, we can compute statistics

Thread Scheduling

- When processes have multiple threads, there are two levels of parallelism
 - Processes
 - Threads
- Scheduling depends on how threads are supported
 - User-level Threads (ULT)
 - Kernel-Level Threads (KLT)

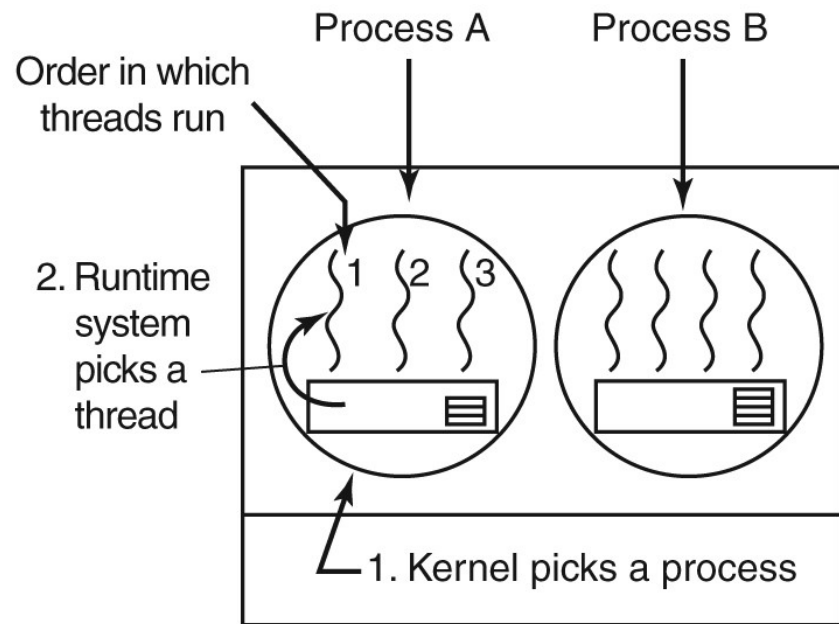
Thread Scheduling

□ User-Level Threads

- Kernel schedules on process level, unaware of threads
- Kernel picks process A, giving it control to its quantum
- Thread scheduler inside process A
 - ▶ Decides which thread to run, say A1
 - ▶ No clock interrupts to multiprogramming threads, so Thread Scheduler cannot interrupt a thread
 - ▶ If A1 uses its quantum, kernel selects another process to run
 - ▶ If A1 blocks, Thread scheduler chose another thread

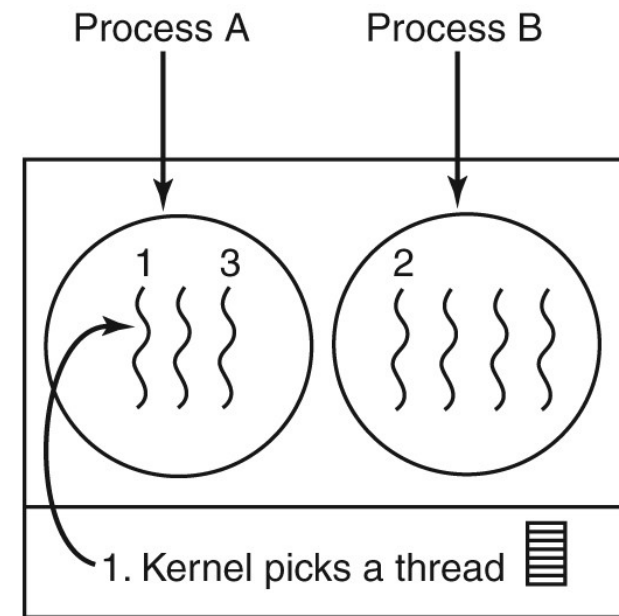
□ Kernel Level Threads

- Kernel picks a thread to run, whichever process it belongs to
- Thread is given a quantum and preempted if uses it, or if blocks
- Another thread is picked by kernel



Possible: A1, A2, A3, A1, A2, A3
 Not possible: A1, B1, A2, B2, A3, B3

(a)



Possible: A1, A2, A3, A1, A2, A3
 Also possible: A1, B1, A2, B2, A3, B3

(b)

(a) Possible scheduling of user-level threads with a 50-msec process quantum and threads that run 5 msec per CPU burst.

(b) Possible scheduling of kernel-level threads with the same characteristics as (a).

ULT vs KLT

- ❑ KLT requires full context switch while ULT performs thread switch taking handful of m/c instructions
- ❑ KLT can be made more complex
 - ❑ Provide Kernel with identity of threads within processes and make decision accordingly
 - ❑ Given 2 threads with same priority, give higher priority to the thread that avoids context switch
- ❑ KLT has the choice to schedule from threads of all processes, maximizing balancing resources
- ❑ ULT can employ application-specific thread schedule, tuning an application better than KLT.
 - ❑ Example: Web server with blocked worker thread and choosing between dispatcher thread and two worker thread: which one to chose? ULT can chose dispatcher so it can then start another worker. KLT does not know

Traditional UNIX Scheduling

- ❑ Designed to provide good **response time** for interactive users while ensuring that low-priority background jobs do not starve
- ❑ **Multilevel Feedback** (Fair-share scheduling) using **round robin** within each of the priority queues.
- ❑ 1-second preemption
- ❑ Priority is based on **process type** and **execution history**
- ❑ Priority divides all processes into fixed **bands** of priority levels, optimizing access to block devices (disks) while responding quickly to systems calls
- ❑ Priorities are recomputed once per second.

- In decreasing order of priority, the **bands** are as follows:
 - Swapper
 - Block I/O device control
 - File manipulation
 - Character I/O device control
 - User processes

Scheduling Formula

$$CPU_j(i) = \frac{CPU_j(i-1)}{2}$$

$$P_j(i) = Base_j + \frac{CPU_j(i)}{2} + nice_j$$

where

$CPU_j(i)$ = measure of processor utilization by process j through interval i

$P_j(i)$ = priority of process j at beginning of interval i ; lower values equal higher priorities

$Base_j$ = base priority of process j

$nice_j$ = user-controllable adjustment factor

- **Base** divide processes into fixed bands of priority levels
- **nice**: allows a user to voluntarily reduce the priority of his process, in order to be nice to the other users. Nobody ever uses it
- Restrictions placed on CPU and **nice** to prevent a process from getting too far away from its base priority band
- within the process band, the use of execution history penalizes CPU-bound process at the expense of I/O-bound process

Example of Traditional UNIX Process Scheduling

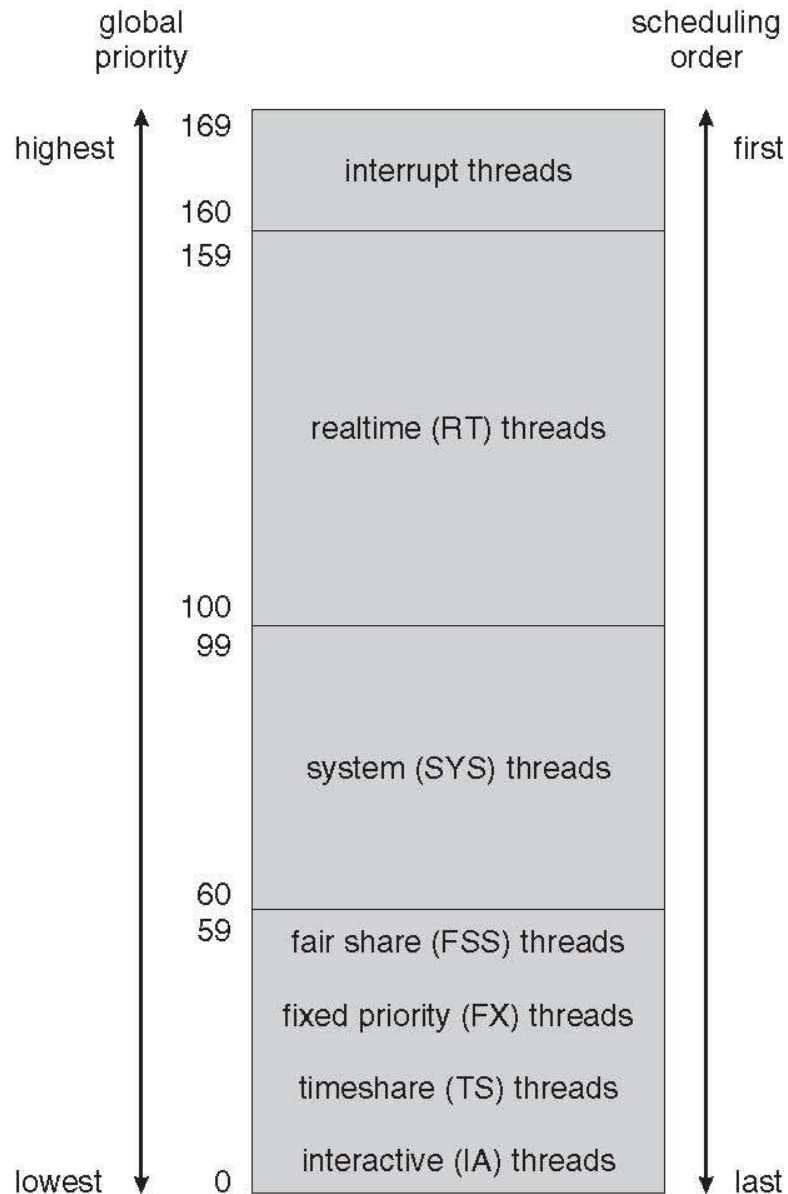
- Processes A, B, and C are created at the same time with base priorities of 60
- The clock interrupts the system 60 times per second and increments a counter for the running process.
- The example assumes that none of the processes block themselves and that no other processes are ready to run.

Time	Process A		Process B		Process C	
	Priority	CPU count	Priority	CPU count	Priority	CPU count
0	60	0	60	0	60	0
		1				
		2				
		•				
		•				
		60				
1	75	30	60	0	60	0
				1		
				2		
				•		
				•		
				60		
2	67	15	75	30	60	0
						1
						2
						•
						•
						60
3	63	7	67	15	75	30
		8				
		9				
		•				
		•				
		67				
4	76	33	63	7	67	15
				8		
				9		
				•		
				•		
				67		
5	68	16	76	33	63	7

Colored rectangle represents executing process

Figure 9.17 Example of Traditional UNIX Process Scheduling

Solaris Scheduling



- kernel threads
- **Six classes** of scheduling
- Default: **time-sharing** based on a **multi-level feedback** queue